

Platform Workflow Surface Review

Compiled: 2026-04-21

Office Workflow Surface Review - 2026-04-21

Why this exists

The current platform has repeatedly drifted toward oversized pages that try to act as queue, creation flow, management console, historical reporting surface, and catch-all lookup UI at the same time.

That pattern is creating worker confusion and operational drag.

The immediate lesson from Estimates and Invoices is that the system should stop assuming one page per domain is automatically correct.

We should design around worker intent, not around dumping all records for a domain onto one surface.

Canonical design rule

When reviewing or rebuilding a page, default to separating these concerns unless there is a strong reason not to:

- **Attention / Queue**
 - things that need action now
 - triage, review, intervention, exceptions, approvals, follow-up
- **Create / Intake**
 - making a new thing
 - fast operational entry, minimal distractions, no historical clutter unless directly needed
- **Management / Search / History**
 - searching by client or other identifier
 - reviewing full context, history, prior actions, details, status, and records

A single page may still own more than one of these only if the workflow is genuinely light and stays understandable in practice.

What this means for estimates

The current idea of one Estimates page doing everything is not worker-friendly.

Better target model

1. Estimate Attention

A dedicated action queue for anything estimate-related that needs attention now, including:

- post-check-in review
- post-wash review
- client rejection
- revision requests
- blocked or exception cases
- optionally ready-to-send items if they still require human review

This surface should answer:

- What needs attention now?
- Why does it need attention?
- What is the next action?

2. Estimate Management

A search-and-context surface for looking up estimate history by client.

This surface should support:

- searching by client/company
- viewing all rugs for that client
- seeing related estimates and statuses
- seeing prior sends, revisions, approvals, rejections, expiry, and communication context

This surface should answer:

- What is the full estimate picture for this client?
- What happened before?
- What is the current state of each rug/estimate?

What this means for invoices

Invoices should follow the same separation.

Better target model

1. Create Invoice

A focused operational workflow:

- search client by name
- load rugs under the client account
- hide rugs already invoiced
- select one or more eligible rugs
- generate invoice
- optionally record payment received immediately and mark paid

This surface should be simple and fast.

It should not be overloaded with historical invoice browsing.

2. Invoice Management

A separate search/history surface:

- search by client
- view invoice history for that client
- inspect invoice details
- see paid/unpaid state
- review older invoices cleanly

Operational identity rule

Workers should not have to rely on awkward generated estimate/invoice strings as the primary way to understand what they are looking at.

Primary visible identity should usually be built from:

- client/company name
- rug number(s)
- rug count
- meaningful status/action context

Generated estimate or invoice codes should remain available as secondary reference identifiers, not the main human-facing anchor.

Broader product direction

The platform should now be reviewed page by page with this question:

> Should this surface actually remain one page, or should it be split by worker intent?

This review should include whether some current surfaces should be kept, split, merged, demoted, or removed entirely.

Suggested review sequence

Priority order should follow operational launch importance, not legacy tab order.

1. Check In / Facility workflow
2. Jobs
3. Estimates
4. Invoices
5. Driver portal
6. Wholesale client portal
7. Office Inbox (deferred, not required for launch)
8. Admin surfaces

For each page, decide:

- Keep as-is
- Keep but simplify

- Split into separate surfaces
- Merge into another domain surface
- Remove

Current decision

The current working vision is:

- build with a stronger product/operations vision first
- do not keep forcing giant all-in-one pages
- prioritize clarity of worker intent over dense all-record dashboards
- pause Office Inbox as a non-essential launch feature and review it later once live operations are stable

Facility Workflow Surface Review - 2026-04-21

Scope

This review covers the current facility-facing surfaces and the related top-priority operational workflow for launch.

Current facility-related surfaces found in the app:

- Check-In (`src/pages/CheckIn.tsx`, `src/components/facility/CheckInLayout.tsx`)
- Pending queue / walk-in selector (`PendingRugsPanel`)
- Check-in form and service flow (`CheckInForm`, `CheckInServiceSelector`, `CheckInPhotoSection`)
- Rug search / rug detail lookup (`RugSearchDialog`, `RugDetailSheet`)
- Delivery Prep (`DeliveryPrepTab`)
- Invoice Generator (`InvoiceGeneratorPanel`)
- Production (`ProductionBoard`)

First principle

Facility should not be treated as one giant mixed workspace.

The facility team does different jobs:

- intake / check-in
- find or inspect a rug
- prep rugs for delivery
- track production / readiness
- sometimes hand off to invoice creation or lookup workflows

Those should not all be mentally mixed together on one surface.

Worker-intent model for facility

The facility domain should be judged by these worker intents:

1. Intake / Check-In

Purpose:

- start or continue intake quickly
- move a pending pickup rug into check-in
- add a walk-in drop-off
- record details, condition, photos, and services
- complete intake with minimal friction

This is the most important launch-critical facility workflow.

2. Find / Inspect Rug

Purpose:

- quickly search by rug number or client
- inspect details for a rug already in the system
- check status or confirm identity

This is supportive, but important.

3. Delivery Prep

Purpose:

- review rugs scheduled for delivery
- confirm what is ready
- organize outgoing rugs reliably

4. Production / Readiness

Purpose:

- understand what is in progress
- know what is ready / blocked
- support the facility team's operational flow without becoming a reporting swamp

Current judgment by surface

1. Check-In

Current shape

- separate top-level page outside the general Facility workspace shell
- left-side pending / walk-in queue on desktop
- form on the right
- mobile toggles between pending and form
- includes search and rug detail access from the shell

Judgment

****Keep, but continue simplifying.****

Why

This is the clearest facility surface right now and it is already closest to a purpose-built worker flow.

The staged rebuild was directionally correct.

Keep

- dedicated Check-In route
- pending pickup / walk-in entry split
- fast-intake emphasis
- separate rug lookup access
- standard-wash fast path

Simplify further

- reduce any remaining conceptual noise in the form
- keep the common path extremely fast
- keep layout stable and calm
- avoid introducing estimate/invoice/history clutter into this page

Rule

Check-In should stay a ****focused intake surface****, not become a mixed operations dashboard.

2. Pending queue / walk-in entry

Current shape

`PendingRugsPanel` currently combines:

- pending rug queue
- queue filter
- walk-in client search
- walk-in rug add flow

Judgment

****Keep, but treat as part of Check-In, not as its own broader workspace concept.****

Why

For intake, this combination is acceptable because it supports one immediate job: selecting what to check in next.

Caveat

If walk-in volume or client search complexity grows, the walk-in flow may later deserve a cleaner dedicated intake-start pattern.

For now it is acceptable if it stays fast and simple.

3. Rug search / rug detail

Current shape

- search dialog accessible from shell
- detail sheet accessible from search or direct selection

Judgment

****Keep as support utilities.****

Why

These are cross-cutting operational tools, not full workflow pages.

They help workers inspect and verify without bloating the main flow.

Rule

Do not turn these into a replacement for proper workflow surfaces.
They should stay lightweight and fast.

4. Delivery Prep

Current shape

- facility workspace panel with date-based selection
- rug list and delivery confirmation behavior
- mixed operational readiness / list management behavior

Judgment

****Keep, but probably simplify.****

Why

This is a real operational need, but it should remain narrowly about outgoing delivery preparation.

Watch-outs

- do not let it become another all-record browser
- prioritize ready/blocked clarity over dense metadata
- keep route/day selection operationally obvious

5. Production

Current shape

- facility production board
- intended to reflect readiness / in-progress state

Judgment

****Keep under review. Likely keep but simplify hard.****

Why

Production state is important, but boards like this often become clutter magnets.
The question is whether it truly helps the facility team do the next action or just exposes system state.

Review question

Should Production remain its own board, or should the most important readiness information be folded into a simpler facility work queue?

This needs a deeper page-specific review later.

6. Invoice Generator inside Facility

Current shape

- facility workspace includes `InvoiceGeneratorPanel`
- search client
- load ready rugs

- exclude already invoiced rugs
- select rugs and generate invoice

Judgment

****Keep the workflow idea, but likely move the product framing.****

Why

The workflow itself is correct and practical:

- search client
- show invoice-eligible rugs
- hide already invoiced rugs
- select rugs
- create invoice

That is good.

But conceptually this is not really a general facility workspace page. It is a dedicated ****Create Invoice**** workflow.

Better target

This should eventually be framed as:

- ****Create Invoice****

not a generic invoice panel living beside unrelated facility tools.

Whether it lives under Facility, Office, or Finance should be decided by actual staff ownership, but the flow itself should remain purpose-built and search-first.

Launch-priority decisions

Must keep and refine now

- Check-In
- intake queue / walk-in selector
- rug search / rug detail support

Keep, but review after core intake is locked

- Delivery Prep
- Production

Keep the workflow, but reframe later

- Invoice Generator as a dedicated create-invoice surface rather than a mixed panel

What facility should NOT become

Facility should not become:

- a dumping ground for every rug-related page
- a giant workspace mixing intake, history, delivery, invoicing, and reporting all together
- a dashboard that forces workers to visually parse everything at once

Current design direction

For launch-priority work, Facility should be organized around the most important worker jobs:

1. **Check-In / Intake**
2. **Production / Ready state**
3. **Delivery Prep**
4. **Search / inspect support utilities**

Any billing/invoice workflow that remains in this domain should behave like a dedicated creation flow, not a giant invoice management page.

Preliminary keep / simplify / split / merge / remove summary

- **Check-In page** → Keep, simplify
- **Pending queue + walk-in entry** → Keep as part of Check-In
- **Rug search dialog** → Keep
- **Rug detail sheet** → Keep
- **Delivery Prep** → Keep, simplify
- **Production board** → Keep under review, likely simplify
- **Invoice Generator panel** → Keep workflow, reframe as dedicated Create Invoice surface later

Next follow-up for facility review

The next deeper facility review should answer:

- Is Production truly helping the next worker action, or just exposing state?
- Should Delivery Prep remain its own page or be narrowed further?
- Where should Create Invoice ultimately live organizationally?
- What remaining elements in Check-In are still concept-heavy and slowing workers down?

Jobs Workflow Surface Review - 2026-04-21

Scope

This review covers the current Jobs surface in ``src/components/office/JobsTab.tsx`` and evaluates whether it matches the intended worker workflow for launch.

Current shape

The current Jobs page is doing several things at once:

- showing grouped jobs by source type (``pickup`` vs ``walkin``)
- exposing search across client, date, route day, rug number, rug type, linked rug tag, and linked services
- allowing expansion/collapse of grouped jobs

- filtering each group by job condition (`all`, `estimate\_open`, `uninvoiced`, `delivered`, `returns`, `attention`)
- editing and creating pickup request items
- editing group-level notes
- opening linked rug context
- mixing operational work tracking with item editing and exception handling

This is already more disciplined than some other surfaces, but it is still carrying too many different worker intents at once.

First judgment

Jobs is important, but it is still trying to be both:

- a working action queue
- and a maintenance/editor surface

That split is where confusion will grow.

Worker-intent model for Jobs

Jobs should be evaluated by the actual reasons a worker uses it.

1. Job Attention / Operations Queue

Purpose:

- see what jobs need attention now
- identify estimate-open work
- identify uninvoiced completed work
- identify delivered/return states that require follow-up
- identify exception cases

This page should answer:

- what needs action now?
- what type of action is needed?
- which client/job/rugs are involved?

2. Job Management / Lookup

Purpose:

- search by client
- inspect grouped job context
- review rug-level details
- review notes and item details
- edit item metadata when needed

This page should answer:

- what is the full job context for this client/request?
- what rugs/items are in it?
- what changed and what still needs work?

3. Job Editing / Maintenance

Purpose:

- create or edit pickup request items
- update metadata safely
- maintain request notes

This is a narrower maintenance function and should not dominate the operational queue.

Current judgment by function

1. Grouped job list

Judgment

****Keep, but refocus.****

Why

Grouping by customer/date/request context is useful and much more operational than raw flat rows.

What should change conceptually

The page should stop trying to show every condition equally.

Jobs should lean toward an ****attention-first**** model, where the most important work rises naturally.

2. Source split (`pickup` vs `walkin`)

Judgment

****Keep for now.****

Why

This reflects real intake origin and is probably operationally meaningful.

Caveat

Only keep it if staff actually think this way while working the page.

If not, it may become internal-system framing rather than worker framing.

3. Job filters (`estimate\_open`, `uninvoiced`, `delivered`, `returns`, `attention`)

Judgment

****Keep the concepts, but likely reorganize them.****

Why

These are useful attention buckets, but they currently behave like embedded filters inside a broad all-purpose screen.

That is better than nothing, but still mentally noisy.

Better target

These should likely become either:

- primary queue modes on an attention page,
 - or clearly elevated summary entry points,
- not just secondary toggles inside a dense page.

4. Inline item editing / creation dialogs

Judgment

****Keep capability, but demote it from the main jobs surface.****

Why

Editing pickup request items is a maintenance task.

It matters, but it should not shape the whole page.

Better target

Job editing should feel like a secondary maintenance function, not the main identity of Jobs.

5. Group notes editing

Judgment

****Keep, but make it contextual.****

Why

Notes are useful, but they should support job handling rather than become another competing sub-workflow.

Core decision

Jobs should probably be split conceptually into:

A. Jobs Attention

A worker-facing operational queue for:

- estimate-open work
- uninvoiced ready work
- return/follow-up work
- delivery/result exceptions
- anything else that truly needs action now

B. Jobs Management

A search-and-context surface for:

- finding client jobs
- reviewing grouped request/rug context
- reading notes
- opening rugs
- performing maintenance edits when needed

This is the same product lesson seen in Estimates and Invoices:

****do not force one page to be queue + editor + history + catch-all search all at once.****

What should remain visible as primary identity

Jobs should be identified primarily by meaningful operational context, such as:

- client name
- scheduled date

- route day where relevant
- rug count
- rug numbers
- current needed action

Raw internal identifiers or hidden system concepts should stay secondary.

Launch-priority judgment

Keep and improve now

- grouped job context
- searchability
- ability to open linked rug context
- attention-oriented filters/concepts

Keep but reframe later

- create/edit dialogs
- maintenance editing behavior
- lower-priority archival conditions

Likely direction

The eventual model should be:

- **Jobs Attention**
- **Jobs Management**

Preliminary keep / simplify / split / merge / remove summary

- **Current grouped Jobs page** → Keep temporarily, but refocus
- **Search** → Keep
- **Grouped context model** → Keep
- **Attention filters** → Keep concept, likely elevate/reorganize
- **Inline maintenance editing** → Keep capability, demote from main identity
- **Single all-purpose Jobs page** → Likely split later into attention + management

Current conclusion

Jobs is important and should stay high priority, but the current page is still doing too much at once.

The safest design direction is not to throw it out, but to progressively separate:

- work needing action now
- from full-context job lookup and maintenance

Estimates Workflow Surface Review - 2026-04-21

Scope

This review covers the current Estimates surface in `src/components/office/EstimatesTab.tsx` and evaluates whether it matches the intended worker workflow for launch.

Current shape

The current Estimates page is trying to do all of the following at once:

- create new estimates from rugs
- display grouped review units
- handle group-level ready/queue/expire actions
- show backend-owned review groups
- load detail rows on demand
- show estimate rows across many statuses
- expose client-approved and client-rejected context
- show batch-send visibility and batch controls
- act as both operational queue and historical management surface

This is exactly the type of all-in-one surface that is creating worker confusion.

First judgment

The current Estimates page is not functionally clean from a worker point of view.

The user-facing concern is correct:

- strange estimate identifiers are not good primary anchors
- grouped review mixed with sent/rejected/expired sections creates a fragmented experience
- the page is too system-shaped and not action-shaped

Worker-intent model for Estimates

Estimates should be separated by what a worker is actually trying to do.

1. Estimate Attention

Purpose:

- see everything estimate-related that needs attention now
- post-check-in review
- post-wash review
- rejection follow-up
- revision requests
- blocked or exception cases
- possibly ready-to-send items awaiting final human action

This page should answer:

- what needs attention now?
- why?
- what action is next?

This should be the primary operational queue.

2. Estimate Management

Purpose:

- search by client
- pull up all rugs for that client
- see associated estimates
- review current and past statuses
- inspect send/review/rejection history
- understand the client's full estimate context cleanly

This page should answer:

- what is going on for this client?
- what happened historically?
- what estimates and rugs are tied together here?

What should NOT happen

Estimate post-check-in review should not be lumped into the same giant page as general estimate history/management/search if that makes the page heavier and harder to work with.

That split is one of the clearest workflow decisions from this review.

Operational identity rule

Workers should not be forced to rely on awkward generated estimate strings as the primary visible identifier.

Primary identity should usually be built from:

- client/company name
- rug number(s)
- rug count
- meaningful action/status context

Estimate number should remain available as a secondary reference, not the main operational anchor.

Current judgment by function

1. Create estimate flow inside the page

Judgment

****Keep capability, but do not let it define the whole surface.****

Why

Creating an estimate is a valid action, but it should not force the entire page to also behave like a review queue and historical browser.

2. Grouped review units

Judgment

****Keep the grouping concept.****

Why

Grouping by client/company is the right operational direction.
This is much better than treating estimate rows as isolated items.

Caveat

The grouped review model belongs in ****Estimate Attention****, not mixed indiscriminately with historical/archive sections.

3. Sent / approved / rejected / expired sections on the same page

Judgment

****Do not keep as primary sections on the same main operational surface.****

Why

These are management/history concerns more than active queue concerns.
Their presence on the same primary screen creates exactly the “messy and confusing” experience the user described.

4. Batch-send visibility and controls

Judgment

****Keep the functionality, but attach it to the attention/review workflow carefully.****

Why

Batch send state is real operational context.
But it should not overwhelm the worker or turn the page into a scheduler control center.

This should likely live with the attention/review workflow or a small office-send queue concept, not as a large secondary dashboard competing with review work.

5. Single all-purpose Estimates page

Judgment

****Do not keep as the long-term model.****

Why

This is the clearest case so far of a surface trying to be queue + review + history + search + send control + creation all in one place.
That is the wrong product shape.

Better target model

A. Estimate Attention

A dedicated queue-oriented surface for:

- needs office review
- needs revision
- rejected / client decision follow-up
- send-ready items requiring action
- other estimate exceptions

Primary identity should be client + rug context, not estimate code.

B. Estimate Management

A search-first client-centered surface for:

- search by client name
- show all rugs under that client
- show associated estimates and statuses
- inspect estimate history, send history, and related details cleanly

This should feel like lookup/context/history, not a queue.

Launch-priority judgment

Keep and improve now

- backend grouping concept
- grouped operational actions
- client-centered grouping and detail loading

Reorganize conceptually as soon as practical

- split attention from management
- demote archive/history statuses from the main action surface
- stop using raw estimate code as the main human-facing identifier

Avoid

- one giant "Estimates" page trying to be everything
- mixing urgent review work with general historical browsing as equal peers

Preliminary keep / simplify / split / merge / remove summary

- ****Current grouped review model**** → Keep
- ****Client/company grouping**** → Keep
- ****Group actions (ready/queue/expire)**** → Keep concept
- ****Estimate creation capability**** → Keep capability
- ****Sent/rejected/expired as large co-equal sections on same page**** → Demote / move to management context
- ****Single all-purpose Estimates page**** → Split into attention + management

Current conclusion

Estimates should not continue as one monolithic page.

The correct design direction is:

- ****Estimate Attention**** for anything needing action now
- ****Estimate Management**** for client-centered search, context, and history

This is a core workflow decision, not a minor UI preference.

Invoices Workflow Surface Review - 2026-04-21

Scope

This review covers the current invoice-related surfaces:

- `src/components/office/InvoicesTab.tsx`
- `src/components/facility/InvoiceGeneratorPanel.tsx`
- related invoice details / creation context

First judgment

The user direction is correct:

invoice creation and invoice management should not be forced into the same giant page or giant always-visible table.

Displaying large invoice lists for everyone by default is not the right worker model.

Invoice work should be centered around:

- creating a new invoice for a client when needed
- looking up invoice history for a client when needed

Worker-intent model for Invoices

1. Create Invoice

Purpose:

- office worker types client name
- sees rugs available to invoice under that account
- already invoiced rugs do not appear
- worker selects one or more rugs
- generates a new invoice
- can optionally record payment received immediately and mark it paid

This should be simple, fast, and search-first.

It should not require browsing unrelated invoice history first.

2. Invoice Management

Purpose:

- search by client
- see invoice history for that client
- inspect invoice details
- see whether invoices are paid or unpaid
- review older invoices and payment context clearly

This should be a clean lookup/history surface, not an operational creation flow.

Current judgment by surface

1. Facility Invoice Generator / create-invoice flow

Judgment

****Keep the workflow concept.****

Why

The actual operational flow is directionally correct:

- search client
- load invoice-eligible rugs
- exclude already invoiced rugs
- select rugs
- generate invoice

That is exactly the worker flow the user described.

Change needed

The important change is conceptual framing.

This should be treated as a dedicated ****Create Invoice**** surface, not as part of a mixed invoice universe.

2. Office Invoices page

Current shape

The current page tries to do many things at once:

- large multi-status invoice list
- status tabs
- date filtering
- client search filtering
- invoice detail sheet
- invoice creation sheet
- status mutation
- PDF download
- payment recording
- credit memo issuing
- thread opening

Judgment

****Too heavy as a single all-purpose page.****

Why

This is another example of queue/history/create/maintenance all being pushed into one surface.

It may be powerful, but it is not the clearest worker experience.

What should change conceptually

Invoice creation should be split away from invoice management.

Management itself should become more client-centered and search-first.

Better target model

A. Create Invoice

A dedicated workflow that starts with client search and only shows invoice-eligible rugs for that client.

Expected behavior:

- search client by name
- show available rugs under account
- do not show already invoiced rugs
- multi-select rugs
- create invoice
- optionally record payment received now

B. Invoice Management

A dedicated lookup/history surface.

Expected behavior:

- search client by name
- load invoices for that client
- view invoice details
- see paid/unpaid state
- review invoice history and payment context

This page should not start by dumping all invoices on workers by default unless there is a very strong operational reason.

What should remain visible as primary identity

Invoices should be understood primarily through:

- client name
- rug numbers / included rugs
- amount
- paid/unpaid state
- issue/due timing

Invoice number should remain available, but should not be the only understandable handle.

Launch-priority judgment

Keep and improve now

- invoice-eligible rug selection flow
- exclusion of already invoiced rugs
- invoice detail inspection
- payment status visibility

Reorganize conceptually as soon as practical

- separate create from management
- make management search-first by client
- stop relying on all-invoice default browsing as the main experience

Preliminary keep / simplify / split / merge / remove summary

- **Invoice creation workflow** → Keep
- **Search client → show invoice-eligible rugs** → Keep
- **Hide already invoiced rugs** → Keep
- **Optional immediate payment capture/recording** → Keep concept
- **Single all-purpose Invoices page** → Split into Create Invoice + Invoice Management
- **Large default all-invoice browsing surface** → Demote

Current conclusion

Invoices should follow the same product rule as Estimates:

- do not make one giant page do everything
- split by worker intent

The correct direction is:

- **Create Invoice**
- **Invoice Management**

That is cleaner, faster, and more usable operationally.

Driver Portal Workflow Surface Review - 2026-04-21

Scope

This review covers the driver-facing operational loading surface, centered on ``src/components/driver/TruckLoadingView.tsx``.

First judgment

Driver workflows should remain highly operational and narrow.

This is not a place for broad management surfaces or heavy mixed workflows.

The driver portal should prioritize:

- what is on today's route
- what is on the truck
- what still needs to be added or confirmed
- what is complete

Current shape

The current truck-loading view does several operational things:

- finds today's active delivery list for the route
- loads confirmed-for-delivery rugs

- shows morning additions available for same-day loading
- groups rugs by client
- tracks loaded-on-truck toggles
- supports add-to-truck behavior for morning additions
- supports final truck checkout/finalization

This is much closer to a real-purpose-built workflow than some office surfaces.

Worker-intent model for Driver Portal

1. Load Truck

Purpose:

- see today's delivery list
- see which rugs belong on the truck
- mark rugs loaded
- add valid morning additions reliably
- finalize truck state

This should be the primary driver workflow.

2. Route Execution / Delivery Confirmation

Purpose:

- support the driver after loading when they are out executing the route
- confirm completion and delivery proof where needed

This may be adjacent, but should remain clearly separate from loading.

Current judgment by function

1. Truck loading flow

Judgment

****Keep.****

Why

This is a good example of a purpose-specific workflow page.

It is focused on one operational job and does not try to be everything.

Important rule

Keep it narrow.

Do not turn it into a broad management surface.

2. Morning additions

Judgment

****Keep, but guard carefully.****

Why

Morning additions are operationally real, but they introduce risk and complexity.

The page already needed idempotency hardening here, which confirms that this area needs

discipline.

Rule

Morning additions should remain explicit and reliable, never ambiguous or duplicate-prone.

3. Grouping by client

Judgment

****Keep.****

Why

This is useful to drivers and maps to how delivery work is actually performed.

Grouping by client is a human-meaningful anchor.

4. Broader driver workflow scope

Judgment

****Do not overload this surface.****

Why

Drivers need simple, reliable, next-action interfaces.

The portal should stay minimal and practical.

Launch-priority judgment

Keep and refine now

- truck loading
- add-to-truck flow
- loaded-state tracking
- finalization flow
- client grouping

Review later

- whether loading and in-route execution should remain together or be more explicitly separated
- whether any additional route context is helpful without increasing cognitive load

Preliminary keep / simplify / split / merge / remove summary

- ****Truck loading view**** → Keep
- ****Morning additions**** → Keep, guard carefully
- ****Client grouping**** → Keep
- ****All-in-one driver management portal**** → Avoid

Current conclusion

Driver Portal should remain a tightly focused operational workflow.

This is one of the clearer product areas because it already has a strong single-job purpose.

The main design rule here is not radical restructuring, but protecting simplicity and reliability.

Wholesale Client Portal Workflow Surface Review - 2026-04-21

Scope

This review covers the current wholesale portal shell and its main client-facing surfaces:

- `src/pages/WholesalePortal.tsx`
- portal rugs
- portal pickups
- portal estimates
- portal invoices
- portal messages
- portal prices

First judgment

The client portal should not mirror the internal app's complexity.
It should feel simpler, clearer, and more client-centered than internal operations surfaces.

The portal should answer a small number of client questions well:

- What rugs do I have with you?
- What pickups are happening?
- Do I have an estimate to review?
- Do I have an invoice to pay or review?
- How do I contact you?

Current shape

The portal currently exposes multiple tabs inside one shell:

- Rugs
- n- Pickups
- Estimates
- Invoices
- Messages
- Prices

This is understandable structurally, but it still risks inheriting internal system complexity if each tab becomes too feature-heavy.

Worker-intent model translated to client intent

Portal design should be organized around client intent rather than internal workflow state.

1. Account Snapshot / Home

Purpose:

- give the client a clear overview

- surface what needs attention now
- reduce the need to hunt across tabs

This is already directionally represented by `PortalAccountSnapshot` and should remain important.

2. View Rugs / Account Status

Purpose:

- see rugs and their status
- understand what is in progress / ready / completed

3. Pickups

Purpose:

- request or review pickup context
- understand upcoming pickup-related activity

4. Estimates

Purpose:

- review estimate items needing client decision
- approve or reject clearly
- understand what the estimate is for

Important note

Portal Estimates should stay focused on client action, not mirror internal estimate management.

5. Invoices

Purpose:

- see invoices
- understand balance status
- inspect invoice contents
- download invoice
- review payment attempt history where relevant

Important note

Portal Invoices should stay focused on client understanding and payment context, not internal finance operations.

6. Messages

Purpose:

- contact the business
- follow ongoing communication threads

Launch note

If messaging is not required for launch, this area can remain secondary rather than consuming major design energy now.

Current judgment by function

1. Portal shell with multiple tabs

Judgment

****Keep, but keep the tabs client-simple.****

Why

Clients can handle multiple tabs if each tab is clear and purpose-limited.

The risk is not the existence of tabs itself, but internal complexity leaking into them.

2. Portal Estimates

Judgment

****Keep, but narrow to client decision workflow only.****

Why

Clients should not see the full complexity of internal estimate lifecycle management.

They only need what is relevant to them.

3. Portal Invoices

Judgment

****Keep, but keep it client-readable and payment-oriented.****

Why

Clients need clarity, not accounting-system density.

4. Portal Messages

Judgment

****Defer if not required for launch.****

Why

This aligns with the broader decision to avoid over-prioritizing messaging before core operational workflows are stable.

Launch-priority judgment

Keep and refine now

- portal account snapshot
- rugs
- pickups
- estimates
- invoices

Lower priority / defer if needed

- portal messages
- anything that mirrors internal complexity without clear client value

Preliminary keep / simplify / split / merge / remove summary

- ****Portal shell**** → Keep

- **Portal account snapshot** → Keep
- **Portal rugs** → Keep
- **Portal pickups** → Keep
- **Portal estimates** → Keep, narrow to client decision role
- **Portal invoices** → Keep, narrow to client payment/history role
- **Portal messages** → Defer if not required for launch
- **Portal prices** → Keep only if it clearly serves a client need and stays simple

Current conclusion

Wholesale Client Portal should remain a simple client-facing product, not a mirror of internal operations.

Its job is clarity and self-service, not exposing every backend concept.